The developer working with the Docker file just has to change the *ENTRYPOINT* to *secretsfoundry* first and then *run* it, followed by the stage in which it is being run. Then the actual script, which was being written earlier as the *ENTRYPOINT*, must be included (as shown in the code snippet above).

This will execute the same logic when Docker starts. You also have to provide the access credentials for AWS Secrets Manager or for Hashicorp Vault once in the deployment infrastructure. And after that, developers can themselves keep pushing code without any manual intervention needed in the development environment for sinking environment variables.

You can also think of integrating SecretsFoundry into your CI/CD pipeline. When it runs, it will print the values of all the environment variables that it can parse out. END

This article is based on a talk given at OSI 2021 by Abhishek Choudhary, co-founder of Ensemble Labs (TrueFoundry).

References

[1] Open sourced code in GitHub: *https://github.com/truefoundry/secretsfoundry*
[2] Documentation: *https://truefoundry.gitbook.io/secretsfoundry/*

By: Abbinaya Kuzhanthaivel

The author is a technology journalist at *OSFY*.

Continued from page no...66

```
import sys
import logging
host_ip = "localhost"
class StdoutFilter(logging.Filter):
    def filter(self, record):
        return record.levelno in (logging.INFO, logging.
ERROR)
logger = logging.getLogger('sampleApp')
logger.setLevel(logging.DEBUG)
handler = logging.StreamHandler(sys.stdout)
handler.addFilter(StdoutFilter())
formatter = logging.Formatter('%(host_ip)s - %(asctime)s -
%(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)
logger.addHandler(handler)
logger = logging.LoggerAdapter(logger, {'host_ip': host_ip})
print(logger)
logger.debug('This is a debug from main module')
logger.info('This is a info from main module')
logger.warning('This is a warning from main module')
logger.error('This is an error from main module')
```

The output is:

```
<LoggerAdapter sampleApp (DEBUG)>
localhost - 2021-11-29 00:26:18,566 - sampleApp - INFO - This
is a info from main module
localhost - 2021-11-29 00:26:18,567 - sampleApp - ERROR -
This is an error from main module
```

Best practices

- Create loggers using the *getlogger* function
- Use appropriate log levels in different regions
- Create module level loggers
- Use rotating file handler
- Avoid opening the same log file multiple times
- Do not pass logger as parameter in class or methods
- Include timestamp in logs; it's preferable to use the standard format called ISO-8601. For example:

```
import logging
logging.basicConfig(format='%(asctime)s %(message)s')
logging.info('Example of logging with ISO-8601 timestamp')
```

As seen, the Python logging module is very flexible, customisable and easy to use. It has many more features such as context based logging, customisable log records, logging to multiple destinations and servers, multiple processes logging to the same file log, etc. The ready code is available in the Python cookbook URL at *https://docs.python.org/3/howto/logging-cookbook.html*.

If you haven't been using logging in your applications yet, it is a good time to begin doing so. You may be surprised at how much it enhances the quality of the code being written. END

References

[1] *https://realpython.com/python-logging/*
[2] *https://docs.python.org/3/howto/logging.html*
[3] *https://docs.python.org/3/howto/logging-cookbook.html*

By: Thangaselvi Arichandrapandian

The author works in a leading bank as AVP. She is a perennial learner and loves the quote: "The more I learn, the more I realise how much I don't know." - *Albert Einstein.*